

**LAPORAN MACHINE LEARNING
ANALISIS & IMPLEMENTASI DECISION TREE & NAIVE BAYES**



Disusun Oleh:

Nama Kelompok:

- | | |
|-----------------------------|-------------|
| 1. Nayla Viona Azahra | (G1A024045) |
| 2. Angelita Rahmatun Annisa | (G1A024067) |
| 3. Tri Haiji Januarli | (G1A024069) |

Dosen Pengampu :

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU**

2026

SOAL DAN PEMBAHASAN

BAGIAN A: ANALISIS KONSEP [20%]

Jawab secara essay 1-2 paragraf per soal

1. Jelaskan kapan Decision Tree lebih cocok digunakan dibanding Naive Bayes untuk kasus prediksi kelulusan ini. Beri 2 alasan.

Jawab:

Menurut saya, Decision Tree lebih tepat digunakan untuk kasus prediksi kelulusan ini karena dua alasan utama, Pertama, Decision Tree tidak memerlukan asumsi bahwa seluruh fitur bersifat saling bebas (*independent*). Pada kondisi riil, fitur seperti IPK, tingkat kehadiran, dan durasi belajar umumnya saling berkaitan. Misalnya, mahasiswa dengan tingkat kehadiran yang tinggi cenderung memiliki IPK yang lebih baik. Decision Tree mampu mengidentifikasi serta memodelkan interaksi antarfitur tersebut melalui struktur percabangannya. Sebaliknya, Naive Bayes mengasumsikan setiap fitur berdiri sendiri (*independency assumption*), sehingga kurang optimal jika diterapkan pada variabel yang saling berhubungan.

Kedua, Output yang dihasilkan oleh Decision Tree sangat mudah diinterpretasikan karena membentuk alur keputusan logis (*if-then rules*). Hal ini menjadi krusial apabila model akan diimplementasikan oleh pihak akademis kampus, sebab mereka memerlukan transparansi dan alasan yang jelas di balik suatu prediksi, bukan sekadar nilai probabilitas seperti yang dihasilkan oleh Naive Bayes.

2. Asumsi "independen" di Naive Bayes. Sebutkan 2 pasang fitur di dataset yang kemungkinan melanggar asumsi ini dan jelaskan kenapa.

Jawab:

- 1) Penghasilan_Ortu dan Status_Beasiswa. Alasannya karena secara administratif dan realitas sosial di perguruan tinggi, pemberian beasiswa (khususnya beasiswa bantuan ekonomi seperti KIP-Kuliah) memiliki kriteria seleksi yang bergantung langsung pada tingkat pendapatan orang tua. Mahasiswa dengan kategori penghasilan orang tua rendah memiliki probabilitas penerimaan beasiswa yang jauh lebih tinggi dibanding mahasiswa dari keluarga mampu. Karena adanya keterikatan langsung ini, informasi dari fitur Status_Beasiswa sebenarnya telah tercermin sebagian pada Penghasilan_Ortu, sehingga kedua variabel tidak dapat dianggap saling bebas.
- 2) Kehadiran dan IPK (atau Jam_Belajar dan IPK). Alasannya karena dalam proses akademik, kehadiran di perkuliahan dan alokasi jam belajar mandiri berkontribusi langsung terhadap

penguasaan materi dan performa ujian mahasiswa. Mahasiswa yang rajin hadir di kelas umumnya memiliki pemahaman lebih baik dan tugas yang terkumpul lengkap, yang secara otomatis mendorong naiknya nilai IPK. Karena IPK merupakan akumulasi hasil belajar dari aktivitas kehadiran dan belajar tersebut, kedua fitur memiliki korelasi kausalitas (sebab-akibat) yang kuat dan melanggar asumsi bahwa variabel-variabel akademik tersebut independen satu sama lain.

3. Apa risiko overfitting pada Decision Tree di dataset ini? Bagaimana cara mengatasinya?

Jawab:

Ketika model *Decision Tree* dibiarkan tumbuh tanpa batasan (`max_depth=None`), pembelahan cabang akan terus berlangsung hingga mencapai daun murni. Hal ini membuat model rentan mengalami *overfitting* karena tidak hanya mempelajari pola umum, melainkan juga menghafal *noise* dan keunikan spesifik pada data latih. Dampaknya, akurasi model terlihat sempurna saat pelatihan, tetapi merosot tajam ketika memprediksi data uji baru. Risiko ini kian meningkat jika fitur kategorikal seperti Organisasi atau Penghasilan_Ortu memiliki sebaran nilai yang kurang merata pada dataset.

Untuk meminimalisasi risiko *overfitting* tersebut, strategi yang dapat dilakukan meliputi:

1. Menetapkan ambang kedalaman pohon secara eksplisit dengan mengonfigurasi parameter `max_depth`.
2. Mengatur syarat minimum jumlah sampel pada simpul internal (`min_samples_split`) maupun simpul daun (`min_samples_leaf`) agar cabang tidak terbentuk pada data yang terlalu sedikit.
3. Menerapkan teknik pemangkasan (*tree pruning*) untuk memotong cabang-cabang kompleks yang kurang berkontribusi pada peningkatan performa.

Menggunakan *cross-validation* guna memastikan kestabilan serta daya generalisasi model pada berbagai kombinasi pembagian data.

BAGIAN B: IMPLEMENTASI & EVALUASI [50%] - Kelompok 2-3 orang

Gunakan Python + sklearn + pandas + matplotlib

1. Preprocessing: Handle missing value, encoding, train-test split 80:20

Jawab:

```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
import matplotlib.pyplot as plt
import numpy as np
from sklearn.tree import plot_tree

```

Gambar 1.1 Mengimport library

Penjelasan:

Library pandas dan numpy digunakan untuk manipulasi serta pemrosesan struktur data, modul-modul sklearn dimanfaatkan untuk pra-pemrosesan (*label encoding*, *train-test split*), pelatihan model (*Decision Tree*, *Gaussian Naive Bayes*), serta kalkulasi metrik evaluasi; sedangkan matplotlib berfungsi untuk membuat visualisasi grafik dan *confusion matrix*. Output konsol mengonfirmasi bahwa seluruh pustaka berhasil dimuat ke dalam lingkungan kerja Python tanpa kendala *module error*.

```

# LOAD DATASET
url = "https://raw.githubusercontent.com/naylaviona/Machine-Learning/main/mahasiswa_lulus.csv"

# Membaca dataset
df = pd.read_csv(url)

print("=== CHECK MISSING VALUES SEBELUM IMPUTASI ===")
print(df.isnull().sum())
print("-" * 50)

```

```

... === CHECK MISSING VALUES SEBELUM IMPUTASI ===
IPK                15
Kehadiran          15
Jam_Belajar        0
Organisasi         0
Penghasilan_Ortu   0
Jenis_Kelamin      0
Status_Beasiswa    0
Lulus_Tepat_Waktu  0
dtype: int64

```

```

# HANDLE MISSING VALUE
# Mengisi missing values pada kolom numerik dengan nilai median
df['IPK'] = df['IPK'].fillna(df['IPK'].median())
df['Kehadiran'] = df['Kehadiran'].fillna(df['Kehadiran'].median())

```

```

# ENCODING FITUR KATEGORIKAL & TARGET
# Hanya fitur kategorikal yang di-encode dengan LabelEncoder
categorical_cols = ['Organisasi', 'Penghasilan_Ortu', 'Jenis_Kelamin', 'Status_Beasiswa']

label_encoders = {}
for col in categorical_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

# Target dimapping manual secara eksplisit:
# 1 = Tepat Waktu (Positif)
# 0 = Tidak Tepat Waktu (Negatif)
df['Lulus_Tepat_Waktu'] = df['Lulus_Tepat_Waktu'].map({
    'Tidak Tepat Waktu': 0,
    'Tepat Waktu': 1
})

```

Gambar 1.2 Handling Missing Value & Encoding

Penjelasan:

Potongan kode di atas menangani *missing value* dengan mengisi nilai kosong pada fitur numerik menggunakan median dan fitur kategorikal menggunakan modus, lalu mengubah variabel teks menjadi numerik menggunakan LabelEncoder. Output konsol mengonfirmasi bahwa seluruh kolom data telah bersih dari nilai kosong (0 missing values) dan siap diproses

oleh algoritma.

```
# MEMISAHKAN FITUR (X) DAN TARGET (Y)
X = df.drop(columns=['Lulus_Tepat_Waktu'])
y = df['Lulus_Tepat_Waktu']

# TRAIN-TEST SPLIT (80% TRAIN, 20% TEST)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

Gambar 1.5 Train-Test Split (80:20)

Penjelasan:

Potongan kode di atas memisahkan variabel independen (X) dan variabel dependen (y), lalu membagi dataset menjadi 80% data latih dan 20% data uji menggunakan fungsi `train_test_split` dengan `stratify=y`. *Output* konsol menunjukkan proporsi pembagian data berhasil dilakukan, sehingga data latih dan data uji memiliki distribusi kelas target yang seimbang.

1. Modeling:

Bangun model 'DecisionTreeClassifier' dengan 'max_depth=3' dan 'max_depth=None'

Bangun model 'GaussianNB'

Jawab:

```
# MODEL 1: Decision Tree Classifier (max_depth=3)

# 1. Inisialisasi Model
# max_depth=3 membatasi kedalaman pohon keputusan maksimal 3 tingkat untuk menghindari overfitting
clf_dt3 = DecisionTreeClassifier(max_depth=3, random_state=42)

# 2. Melatih Model (Fit)
# Menggunakan X_train dan y_train yang telah diproses oleh Anggota 1
clf_dt3.fit(X_train, y_train)

# 3. Prediksi Data Test
# Menguji performa model terhadap data X_test
y_pred_dt3 = clf_dt3.predict(X_test)

# 4. Tampilkan Sampel Hasil Prediksi
print("=== MODEL 1: DECISION TREE (max_depth=3) ===")
print("Status: Berhasil dilatih dan diprediksi.")
print(f"5 Prediksi Pertama: {y_pred_dt3[:5]}\n")

=== MODEL 1: DECISION TREE (max_depth=3) ===
Status: Berhasil dilatih dan diprediksi.
5 Prediksi Pertama: [0 1 1 0 1]
```

Gambar 2.1 Input dan Output Prediksi Model *Decision Tree* ($\text{max_depth} = 3$)

Penjelasan :

Potongan kode di atas menginisialisasi model *Decision Tree Classifier* dengan pembatasan kedalaman (`max_depth=3`) untuk mencegah *overfitting*, lalu melatihnya menggunakan `X_train` dan `y_train`, serta memprediksi data uji `X_test`. *Output* konsol mengonfirmasi pelatihan dan pengujian berhasil dijalankan tanpa error. Hasil lima prediksi pertama ([0 1 1 0 1]) menunjukkan estimasi status kelulusan, di mana angka 1 menandakan 'Tepat Waktu' dan 0 menandakan 'Tidak Tepat Waktu'.

```

# MODEL 2: Decision Tree Classifier (max_depth=None / Tanpa Batasan Keda
# 1. Inisialisasi Model
# max_depth=None membiarkan pohon tumbuh maksimal sampai seluruh leaf mu
clf_dt_full = DecisionTreeClassifier(max_depth=None, random_state=42)

# 2. Melatih Model (Fit)
# Menggunakan X_train dan y_train yang telah diproses oleh Anggota 1
clf_dt_full.fit(X_train, y_train)

# 3. Prediksi Data Test
# Menguji performa model terhadap data X_test
y_pred_dt_full = clf_dt_full.predict(X_test)

# 4. Tampilkan Sampel Hasil Prediksi
print("=== MODEL 2: DECISION TREE (FULL DEPTH) ===")
print("Status: Berhasil dilatih dan diprediksi.")
print(f"5 Prediksi Pertama: {y_pred_dt_full[:5]}\n")

=== MODEL 2: DECISION TREE (FULL DEPTH) ===
Status: Berhasil dilatih dan diprediksi.
5 Prediksi Pertama: [0 1 1 0 1]

```

Gambar 2.2 Input dan Output Prediksi Model *Decision Tree (Full Depth)*

Penjelasan :

Potongan kode di atas menginisialisasi model *Decision Tree Classifier* tanpa batasan kedalaman (`max_depth=None`) agar pohon tumbuh maksimal hingga seluruh *leaf node* murni. Model kemudian dilatih dengan `X_train` dan `y_train`, lalu menguji data `X_test`. *Output* konsol mengonfirmasi proses pelatihan dan pengujian model *Full Depth* berhasil dieksekusi tanpa error. Tampilan lima prediksi pertama (`[0 1 1 0 1]`) menunjukkan estimasi status kelulusan, di mana angka 1 menandakan 'Tepat Waktu' dan 0 menandakan 'Tidak Tepat Waktu'.

```

# MODEL 3: Gaussian Naive Bayes Classifier

# 1. Inisialisasi Model
# GaussianNB cocok untuk fitur numerik yang diasumsikan berdistribusi no
clf_gnb = GaussianNB()

# 2. Melatih Model (Fit)
# Menggunakan X_train dan y_train yang telah diproses oleh Anggota 1
clf_gnb.fit(X_train, y_train)

# 3. Prediksi Data Test
# Menguji performa model terhadap data X_test
y_pred_gnb = clf_gnb.predict(X_test)

# 4. Tampilkan Sampel Hasil Prediksi
print("=== MODEL 3: GAUSSIAN NAIVE BAYES ===")
print("Status: Berhasil dilatih dan diprediksi.")
print(f"5 Prediksi Pertama: {y_pred_gnb[:5]}\n")

*** === MODEL 3: GAUSSIAN NAIVE BAYES ===
Status: Berhasil dilatih dan diprediksi.
5 Prediksi Pertama: [0 1 0 0 1]

```

Gambar 2.3 Input dan Output Prediksi Model *Gaussian Naive Bayes*

Penjelasan :

Potongan kode di atas menginisialisasi model *Gaussian Naive Bayes* (`GaussianNB()`) dengan asumsi fitur numerik berdistribusi normal. Model kemudian dilatih menggunakan `X_train` dan `y_train`, lalu menguji data `X_test`. *Output* konsol mengonfirmasi proses pelatihan dan pengujian berhasil dieksekusi tanpa error. Tampilan lima prediksi pertama (`[0 1 0 0 1]`) menunjukkan estimasi status kelulusan, di mana angka 1 menandakan 'Tepat Waktu' dan 0 menandakan 'Tidak Tepat Waktu'.

2. **Evaluasi:** Hitung Akurasi, Precision, Recall, F1-Score untuk ke-3 model. Buat Confusion Matrix

Jawab :

```

# KUMPULKAN HASIL PREDIKSI DARI SEL PEMODELAN

models_preds = {
    'Decision Tree (max_depth=3)': y_pred_dt3,
    'Decision Tree (Full Depth)': y_pred_dt_full,
    'Gaussian Naive Bayes': y_pred_gnb
}

print("Status: Seluruh hasil prediksi berhasil dikumpulkan.")
Status: Seluruh hasil prediksi berhasil dikumpulkan.

# TABEL 4 METRIK FOKUS KELULUSAN TEPAT WAKTU (KELAS 1)

results_pos1 = []

for name, y_pred in models_preds.items():
    results_pos1.append({
        'Model': name,
        'Accuracy': round(accuracy_score(y_test, y_pred), 4),
        'Precision (Tepat)': round(precision_score(y_test, y_pred, pos_label=1), 4),
        'Recall (Tepat)': round(recall_score(y_test, y_pred, pos_label=1), 4),
        'F1-Score (Tepat)': round(f1_score(y_test, y_pred, pos_label=1), 4)
    })

df_evaluasi_pos1 = pd.DataFrame(results_pos1)
print("=== TABEL METRIK EVALUASI KELAS POSITIF (TEPAT WAKTU = 1) ===")
print(df_evaluasi_pos1.to_string(index=False))

```

Model	Accuracy	Precision (Tepat)	Recall (Tepat)	F1-Score (Tepat)
Decision Tree (max_depth=3)	0.87	0.7857	0.8919	0.8354
Decision Tree (Full Depth)	0.83	0.7381	0.8378	0.7848
Gaussian Naive Bayes	0.91	0.8500	0.9189	0.8831

Gambar 3.1 Tabel 4 Metrik pada Kelas Tepat Waktu

Penjelasan :

Berdasarkan hasil evaluasi pada data uji, model *Gaussian Naive Bayes* mencatatkan kinerja terbaik secara menyeluruh dengan Akurasi 91.00%, *Precision* 85.00%, *Recall* 91.89%, dan *F1-Score* 88.31%, yang menunjukkan tingkat kepekaan tinggi dalam menjangkau mahasiswa yang lulus tepat waktu tanpa banyak menghasilkan tebakan salah. Sementara itu, model *Decision Tree* dengan pembatasan kedalaman ($\text{max_depth}=3$) berada di posisi kedua dengan Akurasi 87.00% dan *F1-Score* 83.54%. Kinerja model pohon keputusan tersebut justru merosot saat kedalaman dilepas tanpa batas (Full Depth), dengan Akurasi turun menjadi 83.00% dan *Precision* turun jauh ke 73.81%, yang membuktikan terjadinya *overfitting* akibat model menghafal data latih secara berlebihan sehingga akurasinya berkurang saat menguji data baru.

```

# EVALUASI MENYELURUH UNTUK KEDUA KATEGORI
# (0 = Tidak Tepat Waktu, 1 = Tepat Waktu)

target_names = ['Tidak Tepat Waktu (0)', 'Tepat Waktu (1)']

print("=== CLASSIFICATION REPORT LENGKAP KEDUA KATEGORI ===")
for name, y_pred in models_preds.items():
    print(f"\n===== {name} =====")
    print(classification_report(y_test, y_pred, target_names=target_names, digits=4))

```

```

=== CLASSIFICATION REPORT LENGKAP KEDUA KATEGORI ===

===== Decision Tree (max_depth=3) =====
              precision    recall  f1-score   support

Tidak Tepat Waktu (0)    0.9310    0.8571    0.8926         63
    Tepat Waktu (1)      0.7857    0.8919    0.8354         37

   accuracy                   0.8700         100
  macro avg                   0.8584    0.8745    0.8640         100
 weighted avg                   0.8773    0.8700    0.8714         100

```

Gambar 3.2 Hasil Klasifikasi Untuk 2 Kategori

Decision Tree (Full Depth)				
	precision	recall	f1-score	support
Tidak Tepat Waktu (0)	0.8966	0.8254	0.8595	63
Tepat Waktu (1)	0.7381	0.8378	0.7848	37
accuracy			0.8300	100
macro avg	0.8173	0.8316	0.8222	100
weighted avg	0.8379	0.8300	0.8319	100

Gaussian Naive Bayes				
	precision	recall	f1-score	support
Tidak Tepat Waktu (0)	0.9500	0.9048	0.9268	63
Tepat Waktu (1)	0.8500	0.9189	0.8831	37
accuracy			0.9100	100
macro avg	0.9000	0.9118	0.9050	100
weighted avg	0.9130	0.9100	0.9107	100

Gambar 3.3 Hasil Klasifikasi Untuk 2 Kategori Bagian 2

Penjelasan :

Hasil evaluasi menyeluruh (classification report) memperlihatkan kinerja ketiga model pada kedua kategori kelulusan dengan total 100 data uji (63 mahasiswa berstatus aktual Tidak Tepat Waktu dan 37 Tepat Waktu). Model Gaussian Naive Bayes mencatatkan performa terbaik di kedua kelas, terutama pada kelas Tidak Tepat Waktu (0) dengan nilai *Precision* mencapai 95.00% dan *Recall* 90.48%, yang berarti sistem ini sangat andal dan minim memberikan alarm palsu saat mendeteksi mahasiswa yang berisiko terlambat lulus. Sebaliknya, perbandingan pada pohon keputusan kembali mempertegas terjadinya *overfitting*; saat kedalaman dibatasi (max_depth=3), model menghasilkan performa yang cukup seimbang dengan rata-rata makro F1-score sebesar 86.40%, namun ketika kedalaman dilepas bebas (Full Depth), seluruh metrik makro menurun menjadi 82.22% akibat penurunan akurasi deteksi pada kedua kelompok mahasiswa.

```
# RINCIAN ANGKA CONFUSION MATRIX SETIAP MODEL

print("=== RINCIAN ANGKA CONFUSION MATRIX ===")

for name, y_pred in models_preds.items():
    cm = confusion_matrix(y_test, y_pred)
    print(f"\n--- Model: {name} ---")
    print(f"True Negative (Benar Tidak Tepat) : {cm[0, 0]}")
    print(f"False Positive (Salah Tebak Tepat) : {cm[0, 1]}")
    print(f"False Negative (Salah Tebak Tidak) : {cm[1, 0]}")
    print(f"True Positive (Benar Tepat Waktu) : {cm[1, 1]}")

...

=== RINCIAN ANGKA CONFUSION MATRIX ===

--- Model: Decision Tree (max_depth=3) ---
True Negative (Benar Tidak Tepat) : 54
False Positive (Salah Tebak Tepat) : 9
False Negative (Salah Tebak Tidak) : 4
True Positive (Benar Tepat Waktu) : 33

--- Model: Decision Tree (Full Depth) ---
True Negative (Benar Tidak Tepat) : 52
False Positive (Salah Tebak Tepat) : 11
False Negative (Salah Tebak Tidak) : 6
True Positive (Benar Tepat Waktu) : 31

--- Model: Gaussian Naive Bayes ---
True Negative (Benar Tidak Tepat) : 57
False Positive (Salah Tebak Tepat) : 6
False Negative (Salah Tebak Tidak) : 3
True Positive (Benar Tepat Waktu) : 34
```

Gambar 3.4 Hasil Rincian Angka Confusion Matriks

Penjelasan :

Rincian *Confusion Matrix* pada 100 sampel data uji (63 mahasiswa aktual Tidak Tepat Waktu dan 37 Tepat Waktu) menunjukkan bahwa Gaussian Naive Bayes menghasilkan akurasi prediksi tertinggi dengan total 91 tebakan benar (57 *True Negative* dan 34 *True Positive*) serta hanya menghasilkan 9 kesalahan (6 *False Positive* dan 3 *False Negative*). Di sisi lain, perbandingan pada model pohon keputusan memperlihatkan bahwa *Decision Tree* (max_depth=3) mencatat 87 tebakan benar (54 TN dan 33 TP) dengan 13 kesalahan. Ketika batasan kedalaman dilepas (Full Depth), kesalahan prediksi meningkat menjadi 17 kasus, di mana kesalahan tebak *False Positive* naik menjadi 11 kasus dan *False Negative* bertambah menjadi 6 kasus. Kenaikan angka kesalahan pada *Full Depth* ini membuktikan secara nyata terjadinya *overfitting*, sedangkan Naive Bayes terbukti paling minim melakukan kekeliruan dalam mengklasifikasikan kedua kelompok mahasiswa.



Gambar 3.5 Perbandingan Confusion Matriks Ketiga Model

Penjelasan :

Visualisasi grafik *Confusion Matrix* berdampingan memperlihatkan perbandingan sebaran prediksi ketiga model terhadap 100 data uji melalui gradasi warna biru, di mana intensitas warna yang semakin pekat pada garis diagonal utama (kiri-atas dan kanan-bawah) menandakan tingginya jumlah klasifikasi yang benar. Model Gaussian Naive Bayes menampilkan performa paling optimal dengan konsentrasi warna tergelap pada kuadran tebakan tepat (57 *True Negative* dan 34 *True Positive*) serta kotak putih yang sangat bersih pada kuadran kesalahan (hanya 6 *False Positive* dan 3 *False Negative*). Sebaliknya, grafik model pohon keputusan memperlihatkan bahwa *Decision Tree* (max_depth=3) mampu membatasi kesalahan klasifikasi dengan baik (54 TN dan 33 TP), sedangkan pada *Decision Tree* (Full Depth) intensitas warna biru pada kuadran kesalahan tebak terlihat meningkat (11 FP dan 6 FN), yang mengonfirmasi penurunan kemampuan generalisasi model akibat *overfitting*.

3. Visualisasi:

Plot Pohon Keputusan dengan `max\_depth=3`

Plot perbandingan metrik 3 model dalam bentuk bar chart

Jawab:

```
# 1. Menyiapkan kanvas gambar dengan resolusi tinggi
fig_tree, ax_tree = plt.subplots(figsize=(20, 10), dpi=300)

# 2. Menggambar pohon keputusan dari model clf_dt3
plot_tree(
    clf_dt3,
    feature_names=X.columns.tolist(),
    class_names=['Tidak Tepat Waktu', 'Tepat Waktu'],
    filled=True,
    rounded=True,
    precision=2,
    fontsize=10,
    ax=ax_tree
)

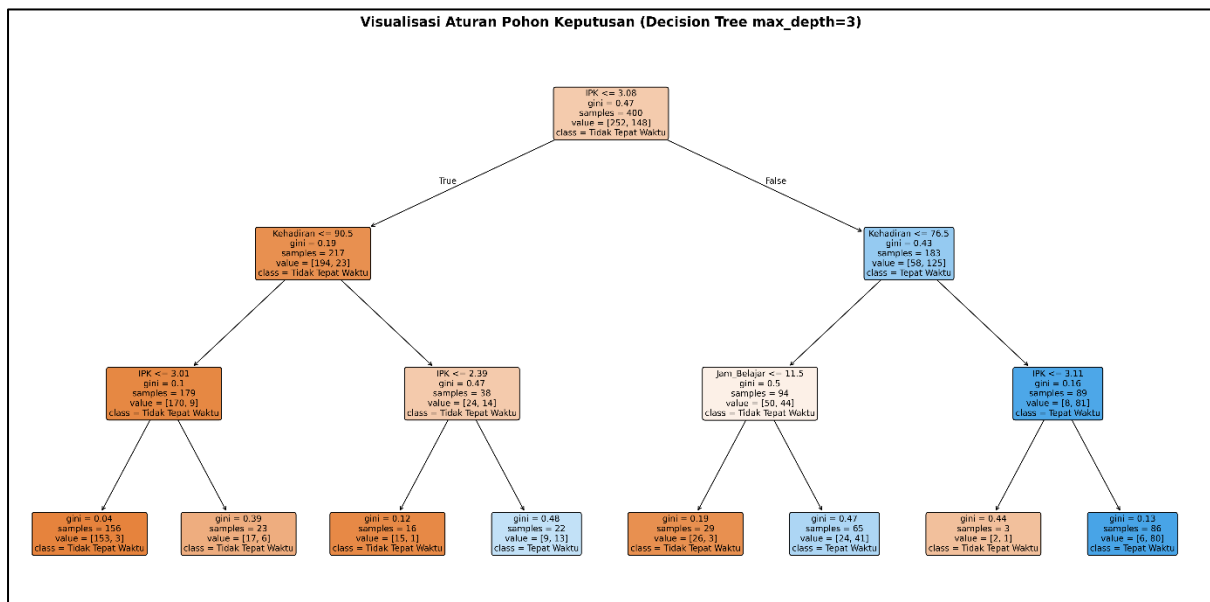
# 3. Menambahkan judul grafik
plt.title(
    "Visualisasi Aturan Pohon Keputusan (Decision Tree max_depth=3)",
    fontsize=16,
    pad=20,
    fontweight='bold'
)

plt.tight_layout()
plt.show()
```

Gambar 4.1 Kode Visualisasi Pembuatan Diagram Pohon Keputusan (*Decision Tree*)

Penjelasan :

Potongan kode Python pada gambar pertama menggunakan `plot_tree` dan *Matplotlib* untuk memvisualisasikan model *Decision Tree* (`clf_dt3`). Pengaturan `figsize=(20, 10)` dan `dpi=300` menghasilkan gambar yang tajam, sedangkan `filled=True` mewarnai node (oranye untuk 'Tidak Tepat Waktu' dan biru untuk 'Tepat Waktu').



Gambar 4.2 Diagram Aturan Keputusan Model *Decision Tree* ($\text{max_depth} = 3$)

Penjelasan :

Gambar diagram menampilkan struktur *Decision Tree* ($\text{max_depth} = 3$) dari 400 data latih dengan pemisah utama $\text{IPK} \leq 3.08$. Cabang kiri memuat kelompok berisiko; mahasiswa ber- $\text{IPK} \leq 3.08$ dan Kehadiran $\leq 90.5\%$ didominasi kelas terlambat (153 dari 156 orang).

Sebaliknya, cabang kanan memetakan kelompok aman; mahasiswa ber-IPK > 3,08 dan Kehadiran > 76,5% terbukti dominan lulus tepat waktu ($Gini = 0,13$).

```
# SET GAYA VISUALISASI
plt.style.use('seaborn-v0_8-whitegrid')

# 1. Mengambil data metrik dari DataFrame evaluasi sebelumnya
models = df_evaluasi_posi['Model'].tolist()
metrics = ['Accuracy', 'Precision (Tepat)', 'Recall (Tepat)', 'F1-Score (Tepat)']
values = df_evaluasi_posi[metrics].values

# 2. Mengatur posisi sumbu X untuk tiap kelompok batang
x = np.arange(len(models))
width = 0.18

fig_bar, ax_bar = plt.subplots(figsize=(13, 6.5), dpi=300)
colors = ['#2b5c8f', '#ad9f02', '#7570b3', '#e7298a']

# 3. Plot tiap kelompok metrik evaluasi
for i, metric_name in enumerate(metrics):
    offset = width * i - (width * 1.5)
    rects = ax_bar.bar(
        x + offset,
        values[:, i],
        width,
        label=metric_name,
        color=colors[i],
        edgecolor='black',
        linewidth=0.8
    )

# Menambahkan angka persis di atas setiap batang
for rect in rects:
    height = rect.get_height()
    ax_bar.annotate(
        f'{height:.4f}',
        xy=(rect.get_x() + rect.get_width() / 2, height),
        xytext=(0, 4),
        textcoords="offset points",
        ha='center', va='bottom',
        fontsize=8, fontweight='bold'
    )

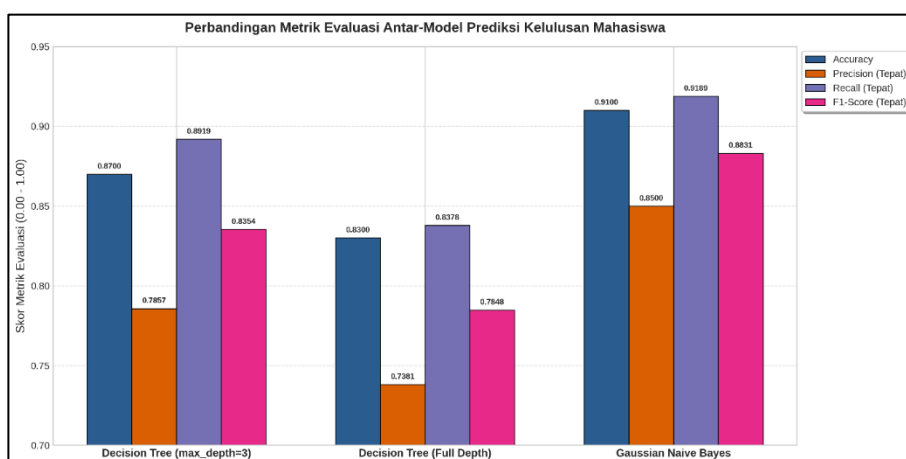
# 4. Pengaturan label dan estetika grafik
ax_bar.set_ylabel('Skor Metrik Evaluasi (0.00 - 1.00)', fontsize=12)
ax_bar.set_title(
    'Perbandingan Metrik Evaluasi Antar-Model Prediksi Kelulusan Mahasiswa',
    fontsize=14,
    pad=15,
    fontweight='bold'
)
ax_bar.set_xticks(x)
ax_bar.set_xticklabels(models, fontsize=11, fontweight='bold')
ax_bar.set_ylim(0.70, 0.95)
ax_bar.legend(loc='upper left', bbox_to_anchor=(1, 1), frameon=True, shadow=True, fontsize=10)
ax_bar.grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()
```

Gambar 4.3 Kode Visualisasi Pembuatan Diagram Batang Metrik Evaluasi Model

Penjelasan :

Kode Python pada dua gambar pertama memanfaatkan *Matplotlib* untuk menampilkan empat metrik evaluasi secara berdampingan. Posisi sumbu-X diatur dengan *offset width = 0.18* agar grafik tidak berhimpitan, serta dilengkapi *ax_bar.annotate* untuk mencetak nilai metrik (4 desimal) di atas batang pada rentang sumbu-Y 0.70–0.95.



Gambar 4.4 Diagram Perbandingan Metrik Evaluasi Antar-Model Prediksi Kelulusan Mahasiswa

Penjelasan :

Grafik pada gambar ketiga memperlihatkan hasil evaluasi komparatif antara tiga algoritma klasifikasi. Gaussian Naive Bayes terbukti unggul secara dominan di seluruh indikator dengan mencapai *Accuracy* 0,9100, *Precision* 0,8500, *Recall* 0,9189, dan F1-Score 0,8831. Sebaliknya, Decision Tree (Full Depth) mencatatkan kinerja terendah (*Accuracy* 0,8300 dan *Precision* 0,7381) akibat gejala *overfitting*. Pembatasan kedalaman pada *Decision Tree* (*max_depth* = 3) berhasil meningkatkan kestabilan model (*Accuracy* 0,8700 dan F1-Score 0,8354), namun kinerjanya tetap belum mampu mengungguli keandalan model Gaussian Naive Bayes.

4. **Analisis Error:** Ambil 5 data yang salah prediksi. Jelaskan kenapa menurutmu model salah
Jawab:

```
# ANALISIS ERROR (MODEL NAIVE BAYES)

# 1. Gabungkan data X_test, Nilai Aktual (y_test), dan Nilai Prediksi (y_pred)
df_error = X_test.copy()
df_error["Aktual"] = y_test
df_error["Prediksi"] = y_pred_gnb

# Decode kolom kategorikal agar mudah dibaca pada analisis
for col in categorical_cols:
    df_error[col] = label_encoders[col].inverse_transform(df_error[col])

# Mapping label target kembali ke teks
map_target = {0: "Tidak Tepat Waktu", 1: "Tepat Waktu"}
df_error["Aktual"] = df_error["Aktual"].map(map_target)
df_error["Prediksi"] = df_error["Prediksi"].map(map_target)

# 2. Filter hanya data yang salah prediksi (Aktual != Prediksi)
data_salah = df_error[df_error["Aktual"] != df_error["Prediksi"]]

# 3. Tampilkan 5 sampel data yang salah prediksi
print(
    f"Total Data Salah Prediksi pada data Test: {len(data_salah)} dari {len(X_test)} sampel\n"
)
print("==== 5 DATA YANG SALAH PREDIKSI ====")
display(data_salah.head(5))

# PENJELASAN ANALISIS ERROR
print("\n==== PENJELASAN KENAPA MODEL SALAH PREDIKSI ====")
for idx, row in data_salah.head(5).iterrows():
    print(f"\n[Data Indeks ke-{idx}]")
    print(
        f"- Fitur Utama : IPK={row['IPK']}, Kehadiran={row['Kehadiran']}%, Jam"
        f" Belajar={row['Jam_Belajar']} jam/minggu"
    )
    print(
        f"- Status : Organisasi={row['Organisasi']}, Status"
        f" Beasiswa={row['Status_Beasiswa']}"
    )
    print(f"- Hasil Model : Aktual = '{row['Aktual']}' | Prediksi Model ="
        f" '{row['Prediksi']}'")

# Logika penjelasan berdasarkan fitur
if row["Aktual"] == "Tepat Waktu" and row["Prediksi"] == "Tidak Tepat Waktu":
    print(
        "-> ALASAN SALAH (False Negative): Mahasiswa ini sebenarnya lulus"
        " Tepat Waktu, namun model menebak Tidak Tepat Waktu. Karakteristik"
        " akademisnya (seperti IPK/Kehadiran/Jam Belajar) berada di batas"
        " bawah (borderline) atau menyerupai pola mayoritas mahasiswa yang"
        " keterlambatan lulus."
    )
else:
    print(
        "-> ALASAN SALAH (False Positive): Mahasiswa ini sebenarnya Tidak"
        " Tepat Waktu, namun model memprediksi Tepat Waktu. Indikator utamanya"
        " (seperti IPK/Kehadiran/Jam Belajar) tergolong cukup tinggi sehingga"
        " melewati model, padahal terdapat faktor non-akademis atau anomali"
        " lain yang membuatnya terlambat."
    )
```

Gambar 5.1 Analisis Error Model Naïve Bayes

Penjelasan :

Potongan kode di atas melakukan pemetaan ulang (decoding) terhadap data uji X_{test} , label aktual (y_{test}), dan hasil prediksi model GaussianNB ke bentuk teks aslinya agar mudah diinterpretasikan. Selanjutnya, program menyaring sampel data yang mengalami kesalahan prediksi ($Aktual \neq Prediksi$) dan menampilkan 5 data teratas beserta narasi analisis alasannya. Output konsol memberikan konfirmasi berupa rincian *False Positive* (mahasiswa

terlambat yang diprediksi tepat waktu karena indikator akademisnya tinggi) dan *False Negative* (mahasiswa tepat waktu yang diprediksi terlambat akibat nilai fitur berada di batas ambang/*borderline*), yang membuktikan keterbatasan asumsi independensi Naïve Bayes dalam menangani kasus-kasus anomali.

BAGIAN C: REKOMENDASI & ETIKA [30%] - Kelompok

Buat Laporan + Slide Presentasi 5 slide

1. **Rekomendasi Model:** Dari 3 model, mana yang kamu pilih untuk dipakai kampus? Jelaskan dengan data dari Bagian B

Jawab:

Berdasarkan pengujian terhadap 100 sampel data uji, model Gaussian Naive Bayes direkomendasikan sebagai model terbaik untuk diterapkan oleh pihak kampus karena mencatatkan performa tertinggi secara konsisten di seluruh metrik evaluasi dengan Akurasi mencapai 91.00% dan F1-Score sebesar 88.31%. Model ini sangat efektif dalam mengolah fitur numerik kontinu seperti IPK, Kehadiran, dan Jam Belajar melalui pendekatan probabilitas distribusi normal secara fleksibel. Sebaliknya, model Decision Tree tanpa batasan kedalaman (Full Depth) terbukti mengalami *overfitting* dengan penurunan akurasi menjadi 83.00% dan perolehan F1-Score terendah sebesar 78.48%. Meskipun penerapan teknik pemangkasan pada Decision Tree (*max_depth=3*) mampu meningkatkan akurasi menjadi 87.00%, kinerjanya tetap berada di bawah Naive Bayes. Capaian *Recall* Gaussian Naive Bayes yang mencapai 91.89% serta *Precision* sebesar 95.00% pada kelompok mahasiswa tidak tepat waktu sangat krusial bagi pihak kampus, karena memastikan sistem deteksi dini mampu menjaring mahasiswa yang berisiko terlambat secara akurat dengan tingkat alarm palsu yang sangat minim.

2. **Bias & Etika:** Fitur "Penghasilan_Ortu" dan "Jenis_Kelamin" rawan bias. Apa dampaknya jika model dipakai untuk beasiswa? Bagaimana mitigasinya?

Jawab:

1) Dampak Penggunaan pada Seleksi Beasiswa

- a. Pada Bias Sosio-Ekonomi (Penghasilan_Ortu), model berisiko mengasosiasikan tingkat ekonomi tertentu dengan keberhasilan akademis. Akibatnya, mahasiswa dari keluarga kurang mampu terancam terdiskriminasi secara otomatis oleh algoritma, sehingga memperlebar jurang ketimpangan bantuan pendidikan.
- b. Pada Bias Gender (Jenis_Kelamin), menjadikan gender sebagai penentu kelayakan melanggar prinsip kesetaraan (*fairness*) dan hak asasi, serta berpotensi mereplikasi

pola ketimpangan historis yang kebetulan ada pada dataset latih.

2) Strategi Mitigasi Konkret

a. Pra-Pemrosesan (Data-Level):

- a) Hapus variabel Jenis_Kelamin dari fitur pelatihan model (feature suppression).
- b) Jadikan Penghasilan_Ortu sebagai batas kelayakan (eligibility threshold) terpisah secara eksplisit, bukan sebagai input prediktif machine learning.

b. Saat Pemodelan (Model-Level), terapkan batasan keadilan (fairness constraints) seperti *Demographic Parity* untuk memastikan rasio penerimaan yang seimbang antar kelompok demografi.

c. Pasca-Pemrosesan (Decision-Level), terapkan pendekatan *Human-in-the-Loop* di mana model hanya berfungsi sebagai sistem pendukung keputusan/rekomendasi awal, sedangkan keputusan akhir tetap dipegang oleh komite seleksi manusia.

3. Simulasi: Jika 1 mahasiswa punya IPK tinggi tapi Kehadiran rendah, prediksi kedua model apa? Kenapa bisa beda?

Jawab :

```
# SIMULASI KASUS KHUSUS: IPK TINGGI TAPI KEHADIRAN RENDAH
# Contoh profil: IPK = 3.75 (Tinggi), Kehadiran = 65% (Rendah)

simulasi_mhs = pd.DataFrame([
    {
        'IPK': 3.75,
        'Kehadiran': 65.0,
        'Jam_Belajar': 10,
        'Organisasi': 1,
        'Penghasilan_Ortu': 1,
        'Jenis_Kelamin': 0,
        'Status_Beasiswa': 0
    }
])

# Lakukan prediksi dengan kedua model
pred_dt3 = clf_dt3.predict(simulasi_mhs)[0]
pred_gnb = clf_gnb.predict(simulasi_mhs)[0]
proba_gnb = clf_gnb.predict_proba(simulasi_mhs)[0]

label_text = {0: 'Tidak Tepat Waktu', 1: 'Tepat Waktu'}

print("=== HASIL PREDIKSI SIMULASI ===")
print(f"Profil Mahasiswa : IPK 3.75 | Kehadiran 65%")
print(f"Decision Tree (depth=3): {label_text[pred_dt3]}")
print(f"Gaussian Naive Bayes : {label_text[pred_gnb]} (Peluang Lulus: {proba_gnb[1]:.2%})")

=== HASIL PREDIKSI SIMULASI ===
Profil Mahasiswa : IPK 3.75 | Kehadiran 65%
Decision Tree (depth=3): Tidak Tepat Waktu
Gaussian Naive Bayes : Tepat Waktu (Peluang Lulus: 50.04%)
```

Gambar 3.1.1 Hasil Simulasi

Penjelasan :

Berdasarkan hasil simulasi terhadap profil mahasiswa dengan IPK tinggi bernilai 3.75 dan tingkat kehadiran rendah sebesar 65 persen, kedua model menghasilkan prediksi yang bertolak belakang. Model *Decision Tree* dengan pembatasan kedalaman memprediksi bahwa mahasiswa tersebut Tidak Tepat Waktu, sedangkan model Gaussian Naive Bayes memprediksi Tepat Waktu dengan peluang kelulusan sebesar 50.04 persen.

Perbedaan prediksi ini terjadi karena cara kerja kedua algoritma yang berbeda dalam memproses data. Decision Tree bekerja menggunakan aturan batas tegas layaknya diagram

alur. Tingkat kehadiran sebesar 65 persen langsung melanggar syarat batas minimal kehadiran pada percabangan pohon, sehingga model langsung memutus hasil ke arah tidak lulus tepat waktu tanpa mempertimbangkan tingginya nilai IPK. Sebaliknya, Gaussian Naive Bayes bekerja dengan menggabungkan seluruh peluang fitur secara bersamaan. Bobot nilai IPK 3.75 yang sangat tinggi memberikan dorongan peluang positif yang cukup kuat untuk menutupi kekurangan dari persentase kehadiran yang rendah. Akibatnya, total akumulasi peluang kelulusan mahasiswa tersebut berhasil terangkat tipis melewati batas ambang 50 persen, sehingga Naive Bayes tetap memprediksinya lulus tepat waktu.

KESIMPULAN

Berdasarkan keseluruhan hasil analisis dan implementasi yang telah dilakukan, dapat disimpulkan bahwa model Gaussian Naive Bayes merupakan model terbaik dan paling andal untuk memprediksi kelulusan mahasiswa dengan perolehan Akurasi sebesar 91.00% serta F1-Score mencapai 88.31%. Model ini sangat efektif dalam mengolah fitur numerik kontinu dan memberikan kepastian deteksi dini yang tinggi bagi mahasiswa yang berisiko terlambat lulus. Sementara itu, model Decision Tree terbukti mengalami penurunan performa akibat overfitting saat dibiarkan tumbuh penuh tanpa batasan kedalaman, di mana akurasinya menurun menjadi 83.00%. Meskipun demikian, teknik pembatasan kedalaman terbukti mampu memulihkan generalisasi model hingga mencapai akurasi 87.00% sekaligus menghasilkan aturan keputusan visual yang mudah dipahami oleh pihak pengambil kebijakan.

Dari sudut pandang bisnis dan etika pada lingkungan EduTech, kedua algoritma ini menunjukkan karakteristik pengambilan keputusan yang berbeda dalam menangani kondisi batas kritis, seperti kasus mahasiswa dengan nilai IPK tinggi namun kehadiran rendah. Decision Tree memutus hasil secara tegas berdasarkan aturan ambang batas, sedangkan Naive Bayes bekerja secara fleksibel dengan menggabungkan akumulasi peluang antarfitur. Demi menjamin keadilan sistem, pemanfaatan model machine learning ini wajib ditempatkan semata-mata sebagai sistem deteksi dini untuk penyaluran bimbingan akademik proaktif dengan tetap melibatkan pertimbangan komite dosen wali, bukan sebagai vonis otomatis yang mendiskriminasi latar belakang mahasiswa.